

After the works of

https://github.com/animesh1012/machineLearning/blob/main/Music_Genre_Classification/Train_Music_Genre_Classifier.ipynb

```
import os
import sys
import librosa
import librosa.display
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPool2D, Flatten, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical
from tensorflow.image import resize
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
```

```
print("Python version:", sys.version)
print("Librosa version:", librosa.__version__)
print("Numpy version:", np.__version__)
print("Tensorflow version:", tf.__version__)
print("Matplotlib version:", matplotlib.__version__)
print("Seaborn version:", sns.__version__)
```

```
Python version: 3.10.12 | packaged by Anaconda, Inc. | (main, Jul 5 2023, 19:01:18) [MSC v.1916 64 bit (AMD64)]
Librosa version: 0.10.2.post1
Numpy version: 1.25.2
Tensorflow version: 2.14.0
Matplotlib version: 3.7.1
Seaborn version: 0.13.1
```

```
# Load and preprocess audio data
def load_and_preprocess_data(data_dir, classes, target_shape=(150, 150)):
    data = []
    labels = []

    for i_class, class_name in enumerate(classes):
        class_dir = os.path.join(data_dir, class_name)
        print("Processing--", class_name)
        for filename in os.listdir(class_dir):
            if filename.endswith(".wav"):
                file_path = os.path.join(class_dir, filename)

                duration = librosa.get_duration(path=file_path)

                if duration > 90: # longer than 1.5 minutes
                    audio_data, sample_rate = librosa.load(
                        file_path,
                        sr=None,
                        offset=30,
                        duration=60
                    )
                else:
                    audio_data, sample_rate = librosa.load(file_path, sr=None)
                # Perform preprocessing (e.g., convert to Mel spectrogram and resize)
                # Define the duration of each chunk and overlap
                chunk_duration = 4 # seconds
                overlap_duration = 2 # seconds

                # Convert durations to samples
                chunk_samples = chunk_duration * sample_rate
                overlap_samples = overlap_duration * sample_rate

                # Calculate the number of chunks
                num_chunks = int(np.ceil((len(audio_data) - chunk_samples) / (chunk_samples - overlap_samples))) + 1

                # Iterate over each chunk
                for i in range(num_chunks):
                    # Calculate start and end indices of the chunk
                    start = i * (chunk_samples - overlap_samples)
                    end = start + chunk_samples

                    # Extract the chunk of audio
                    chunk = audio_data[start:end]

                    mel_spectrogram = librosa.feature.melspectrogram(y=audio_data, sr=sample_rate)
                    mel_spectrogram = resize(np.expand_dims(mel_spectrogram, axis=-1), target_shape)
```

```

        data.append(mel_spectrogram)
        labels.append(i_class)

    return np.array(data), np.array(labels)

```

```

classes = {
    'blues': 0, 'classical': 1, 'country': 2, 'disco': 3, 'hiphop': 4,
    'jazz': 5, 'metal': 6, 'pop': 7, 'reggae': 8, 'rock': 9
}

```

```
data, labels = load_and_preprocess_data("data/genresWAV", classes)
```

```

Processing-- blues
Processing-- classical
Processing-- country
Processing-- disco
Processing-- hiphop
Processing-- jazz
Processing-- metal
Processing-- pop
Processing-- reggae
Processing-- rock

```

```

np.save('Gdata_spec_final.npy', data)
np.save('Glabels_spec_final.npy', labels)

```

```

labels = to_categorical(labels, num_classes=len(classes)) # Convert labels to one-hot encoding
labels

```

```

array([[1., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.]], dtype=float32)

```

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(data, labels, test_size=0.2, random_state=42)

```

```
X_train[0].shape
```

```
(150, 150, 1)
```

```
X_train.shape, y_train.shape
```

```
((11992, 150, 150, 1), (11992, 10))
```

```

model = tf.keras.models.Sequential()
model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=X_train[0].shape))
model.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
model.add(MaxPool2D(pool_size=2, strides=2))
model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=64, kernel_size=3, activation='relu'))
model.add(MaxPool2D(pool_size=2, strides=2))
model.add(Conv2D(filters=128, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=128, kernel_size=3, activation='relu'))
model.add(MaxPool2D(pool_size=2, strides=2))
model.add(tf.keras.layers.Dropout(0.3))
model.add(Conv2D(filters=256, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=256, kernel_size=3, activation='relu'))
model.add(MaxPool2D(pool_size=2, strides=2))
model.add(Conv2D(filters=512, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=512, kernel_size=3, activation='relu'))
model.add(MaxPool2D(pool_size=2, strides=2))
model.add(Dropout(0.3))
model.add(Flatten())
model.add(Dense(units=1200, activation='relu'))
model.add(Dropout(0.45))

model.add(Dense(units=len(classes), activation='softmax'))

model.summary()

```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	320

conv2d_1 (Conv2D)	(None, 148, 148, 32)	9248
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_2 (Conv2D)	(None, 74, 74, 64)	18496
conv2d_3 (Conv2D)	(None, 72, 72, 64)	36928
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_4 (Conv2D)	(None, 36, 36, 128)	73856
conv2d_5 (Conv2D)	(None, 34, 34, 128)	147584
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
dropout (Dropout)	(None, 17, 17, 128)	0
conv2d_6 (Conv2D)	(None, 17, 17, 256)	295168
conv2d_7 (Conv2D)	(None, 15, 15, 256)	590080
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 256)	0
conv2d_8 (Conv2D)	(None, 7, 7, 512)	1180160
conv2d_9 (Conv2D)	(None, 5, 5, 512)	2359808
max_pooling2d_4 (MaxPooling2D)	(None, 2, 2, 512)	0
dropout_1 (Dropout)	(None, 2, 2, 512)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 1200)	2458800
dropout_2 (Dropout)	(None, 1200)	0
dense_1 (Dense)	(None, 10)	12010

=====

Total params: 7182458 (27.40 MB)
 Trainable params: 7182458 (27.40 MB)
 Non-trainable params: 0 (0.00 Byte)

```
model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])
```

```
batch_size = 128
callback = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', min_delta=0, patience=8, verbose=0, mode='auto',
    baseline=None, restore_best_weights=True)

history = model.fit(X_train,y_train ,validation_data=(X_test,y_test),
    epochs=30,
    verbose=1,
    callbacks = [callback],batch_size=batch_size)
```

```
Epoch 2/30
94/94 [=====] - 1078s 11s/step - loss: 1.6270 - accuracy: 0.4338 - val_loss: 1.2275 - val_accuracy: 0.4338
Epoch 3/30
94/94 [=====] - 1130s 12s/step - loss: 1.0967 - accuracy: 0.6171 - val_loss: 0.6861 - val_accuracy: 0.6171
Epoch 4/30
94/94 [=====] - 1089s 12s/step - loss: 0.7321 - accuracy: 0.7496 - val_loss: 0.4528 - val_accuracy: 0.7496
Epoch 5/30
94/94 [=====] - 1089s 12s/step - loss: 0.3996 - accuracy: 0.8683 - val_loss: 0.1893 - val_accuracy: 0.8683
Epoch 6/30
94/94 [=====] - 1083s 12s/step - loss: 0.2608 - accuracy: 0.9165 - val_loss: 0.1720 - val_accuracy: 0.9165
Epoch 7/30
94/94 [=====] - 1092s 12s/step - loss: 0.1498 - accuracy: 0.9545 - val_loss: 0.0335 - val_accuracy: 0.9545
Epoch 8/30
94/94 [=====] - 1097s 12s/step - loss: 0.0904 - accuracy: 0.9755 - val_loss: 0.0165 - val_accuracy: 0.9755
Epoch 9/30
94/94 [=====] - 1099s 12s/step - loss: 0.0796 - accuracy: 0.9806 - val_loss: 0.0147 - val_accuracy: 0.9806
Epoch 10/30
94/94 [=====] - 1091s 12s/step - loss: 0.0728 - accuracy: 0.9804 - val_loss: 0.0120 - val_accuracy: 0.9804
Epoch 11/30
94/94 [=====] - 1095s 12s/step - loss: 0.0319 - accuracy: 0.9932 - val_loss: 0.0067 - val_accuracy: 0.9932
```

```
Epoch 13/30
```

```

94/94 [=====] - 1102s 12s/step - loss: 0.0270 - accuracy: 0.9942 - val_loss: 0.0102 - val_accurac
Epoch 14/30
94/94 [=====] - 1097s 12s/step - loss: 0.0281 - accuracy: 0.9929 - val_loss: 0.0013 - val_accurac
Epoch 15/30
94/94 [=====] - 1094s 12s/step - loss: 0.0305 - accuracy: 0.9921 - val_loss: 0.0032 - val_accurac
Epoch 16/30
94/94 [=====] - 1098s 12s/step - loss: 0.0165 - accuracy: 0.9955 - val_loss: 0.0014 - val_accurac
Epoch 17/30
94/94 [=====] - 1103s 12s/step - loss: 0.0097 - accuracy: 0.9981 - val_loss: 0.0012 - val_accurac
Epoch 18/30
94/94 [=====] - 1057s 11s/step - loss: 0.0089 - accuracy: 0.9982 - val_loss: 0.0018 - val_accurac
Epoch 19/30
94/94 [=====] - 996s 11s/step - loss: 0.0149 - accuracy: 0.9963 - val_loss: 0.0019 - val_accuracy
Epoch 20/30
94/94 [=====] - 993s 11s/step - loss: 0.0223 - accuracy: 0.9947 - val_loss: 0.0044 - val_accuracy
Epoch 21/30
94/94 [=====] - 965s 10s/step - loss: 0.0154 - accuracy: 0.9962 - val_loss: 0.0040 - val_accuracy
Epoch 22/30
94/94 [=====] - 951s 10s/step - loss: 0.0063 - accuracy: 0.9982 - val_loss: 7.1242e-04 - val_accu
Epoch 23/30
94/94 [=====] - 929s 10s/step - loss: 0.0067 - accuracy: 0.9982 - val_loss: 0.0042 - val_accuracy
Epoch 24/30
94/94 [=====] - 915s 10s/step - loss: 0.0098 - accuracy: 0.9976 - val_loss: 0.0016 - val_accuracy
Epoch 25/30
94/94 [=====] - 922s 10s/step - loss: 0.0244 - accuracy: 0.9937 - val_loss: 0.0067 - val_accuracy
Epoch 26/30
94/94 [=====] - 911s 10s/step - loss: 0.0208 - accuracy: 0.9950 - val_loss: 0.0030 - val_accuracy
Epoch 27/30
94/94 [=====] - 923s 10s/step - loss: 0.0125 - accuracy: 0.9967 - val_loss: 0.0146 - val_accuracy
Epoch 28/30
94/94 [=====] - 939s 10s/step - loss: 0.0462 - accuracy: 0.9853 - val_loss: 0.0018 - val_accuracy
Epoch 29/30
94/94 [=====] - 1128s 12s/step - loss: 0.0057 - accuracy: 0.9986 - val_loss: 0.0016 - val_accurac
Epoch 30/30
94/94 [=====] - 1047s 11s/step - loss: 0.0070 - accuracy: 0.9970 - val_loss: 0.0000 - val_accuracy

```

```

import json
with open('history.json','w') as f:
    json.dump(history.history,f)

```

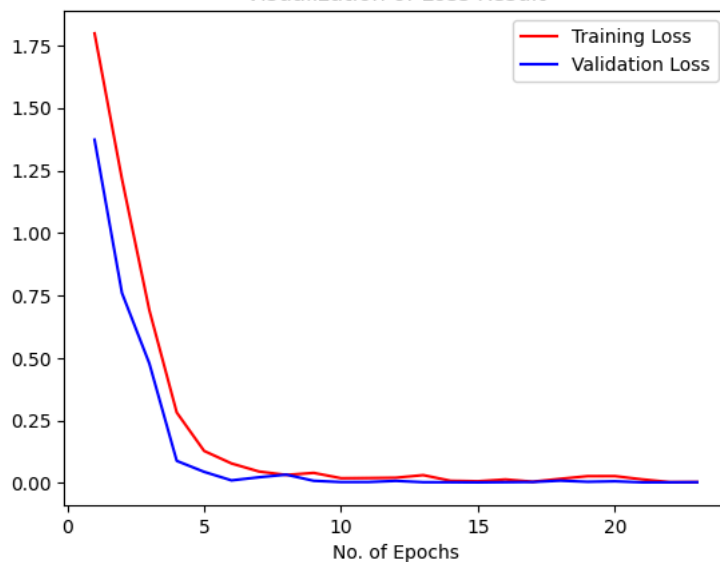
```

#Visualization of Loss
epochs = [i for i in range(1,24)]
plt.plot(epochs,history.history['loss'],color='red',label='Training Loss')
plt.plot(epochs,history.history['val_loss'],color='blue',label='Validation Loss')
plt.xlabel('No. of Epochs')
plt.title('Visualization of Loss Result')
plt.legend()
plt.show()

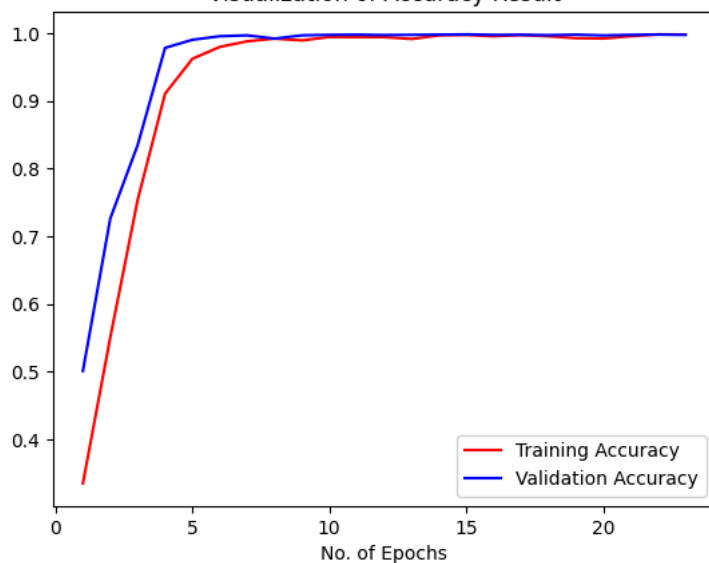
#Accuracy Visualization
plt.plot(epochs,history.history['accuracy'],color='red',label='Training Accuracy')
plt.plot(epochs,history.history['val_accuracy'],color='blue',label='Validation Accuracy')
plt.xlabel('No. of Epochs')
plt.title('Visualization of Accuracy Result')
plt.legend()
plt.show()

```

Visualization of Loss Result



Visualization of Accuracy Result



```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

accuracy = accuracy_score(y_true, y_pred)

precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")
```

```
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

y_pred=model.predict(X_test)
round_off=np.round(y_pred)
cm = confusion_matrix(y_test.argmax(axis=1), round_off.argmax(axis=1))

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

fig, ax = plt.subplots(figsize=(20, 20))
disp.plot(ax=ax)
```

```
plt.title("Confusion Matrix")
```

```
220/220 [=====] - 160s 660ms/step
```

